

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



BLM 4522- Ağ Tabanlı Paralel Dağıtım Sistemleri

PROJE 2 RAPORU

Veri Temizleme ve ETL Süreçleri Tasarımı

Emine Bülbül

21290189

06 Haziran 2026

<https://github.com/eminebulbul/FinalHospitalDB>

İÇİNDEKİLER

1. PROJENİN AMACI VE KAPSAMI	3
2. KULLANILAN TEKNOLOJİLER VE SİSTEM MİMARISI	3
3. ETL (EXTRACT, TRANSFORM, LOAD) SÜRECİ VE METODOLOJİLER.....	4
3.1. HAM (KIRLI) VERİ HAVUZUNUN OLUŞTURULMASI (EXTRACT SİMÜLASYONU)	4
3.2. VERİ DÖNÜŞTÜRME VE TEMİZLEME ADIMLARI (TRANSFORM)	4
3.3. TEMİZLENMİŞ VERİNİN OPERASYONEL TABLOYA AKTARIMI (LOAD).....	5
4. VERİ KALİTESİ VE SÜREÇ RAPORLAMASI	5
4.1. ANALİTİK VERİ KALİTESİ RAPORUNUN (VIEW) TASARLANMASI.....	5
5. SONUÇ VE KAZANIMLAR	6

1. Projenin Amacı ve Kapsamı

Sağlık bilgi sistemlerine dış kaynaklardan (farklı klinikler, laboratuvarlar, eski sistemler veya manuel girişler) gelen veriler genellikle standart dışı, eksik veya hatalı formatlarda olabilmektedir. "Garbage in, garbage out" (Çöp giren, çöp çıkar) prensibi gereği, bu kirli verilerin doğrudan canlı (Production) hastane veritabanına alınması, veri bütünlüğünü bozmakta ve operasyonel hatalara yol açmaktadır.

Bu projenin amacı, dış kaynaklardan gelen yapılandırılmamış veya hatalı verileri filtrelemek, standartlaştırmak ve güvenli bir şekilde operasyonel tablolara aktarmak için bir ETL (Extract, Transform, Load) mimarisi tasarlamaktır. Çalışma kapsamında, SQL tabanlı veri temizleme fonksiyonları kurgulanmış ve sürecin şeffaflığını sağlamak adına bir Veri Kalitesi Denetim Raporu oluşturulmuştur.

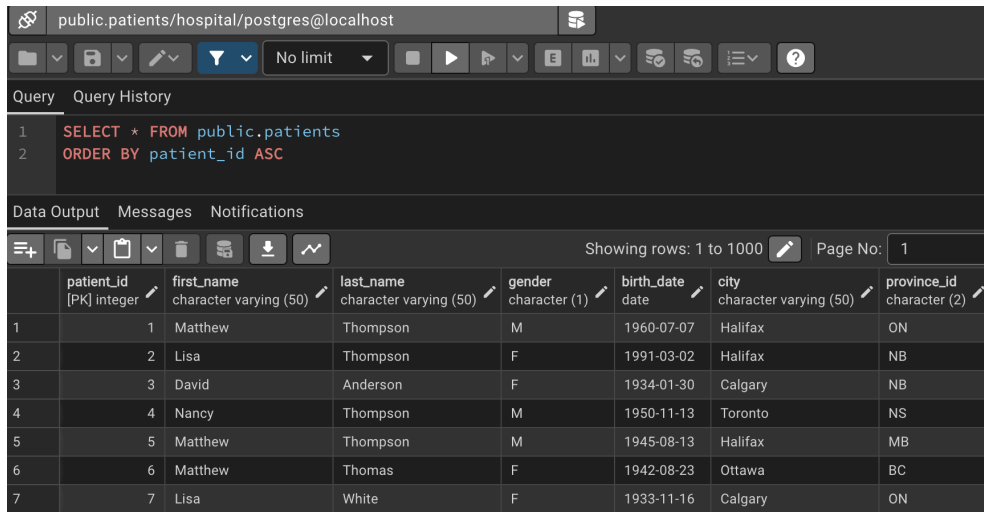
2. Kullanılan Teknolojiler ve Sistem Mimarisi

Proje, veri manipülasyonu ve dönüşüm süreçlerini veritabanı motoru (Database Engine) seviyesinde gerçekleştirilerek maksimum performans elde etmeyi hedeflemiştir.

- **Veritabanı Motoru:** PostgreSQL 15+ (Veri manipülasyonu ve View oluşturma işlemleri için)
- **Arayüz (GUI):** pgAdmin 4 (SQL scriptlerinin çalıştırılması ve sonuçların analizi)
- **Kullanılan Veri Dönüşüm Fonksiyonları:** INITCAP, TRIM, COALESCE, NULLIF, CASE WHEN

```
CleanTable.sql
1 CREATE TABLE clean_patients (
2     patient_id SERIAL PRIMARY KEY,
3     formatted_name VARCHAR(100),
4     contact_number VARCHAR(20),
5     valid_birth_date DATE,
6     blood_type VARCHAR(5)
7 );
```

Şekil 1. Temiz Tablo



The screenshot shows the pgAdmin 4 interface. At the top, the connection is 'public.patients/hospital/postgres@localhost'. Below the toolbar, the 'Query' tab is active, showing a SQL query: 'SELECT * FROM public.patients ORDER BY patient_id ASC'. The 'Data Output' tab is also visible, showing a table with 7 rows and 8 columns. The columns are: patient_id (integer), first_name (character varying (50)), last_name (character varying (50)), gender (character (1)), birth_date (date), city (character varying (50)), and province_id (character (2)). The table contains the following data:

	patient_id [PK] integer	first_name character varying (50)	last_name character varying (50)	gender character (1)	birth_date date	city character varying (50)	province_id character (2)
1	1	Matthew	Thompson	M	1960-07-07	Halifax	ON
2	2	Lisa	Thompson	F	1991-03-02	Halifax	NB
3	3	David	Anderson	F	1934-01-30	Calgary	NB
4	4	Nancy	Thompson	M	1950-11-13	Toronto	NS
5	5	Matthew	Thompson	M	1945-08-13	Halifax	MB
6	6	Matthew	Thomas	F	1942-08-23	Ottawa	BC
7	7	Lisa	White	F	1933-11-16	Calgary	ON

Şekil 2. DB Ana tablo

3. ETL (Extract, Transform, Load) Süreci ve Metodolojiler

3.1. Ham (Kirli) Veri Havuzunun Oluşturulması (Extract Simülasyonu)

Teorik Altyapı: Dış sistemlerden alınan verilerin (Extract), canlı veritabanına temas etmeden önce bekletildiği geçici bir karşılama alanıdır (Staging Area).

Teknik Uygulama: Sisteme raw_patients adında bir havuz tablo oluşturulmuş ve içerisine kasıtlı olarak formatı bozuk isimler, eksik telefon numaraları ve gelecekte yer alan mantıksız doğum tarihleri barındıran "kirli" veriler eklenerek senaryo simüle edilmiştir.

Dış sistemlerden gelecek formatı bozuk veya eksik ham verileri karşılamak üzere tasarlanan raw_patients (Staging Area / Bekleme Havuzu) tablosunun tanımlanması.

```
CREATE TABLE raw_patients (  
    raw_id SERIAL PRIMARY KEY,  
    full_name VARCHAR(100),  
    phone_number VARCHAR(20),  
    birth_date DATE,  
    blood_type VARCHAR(5)  
);
```

Şekil 3. Kirli Veri Tablosu Oluşturma

Veri temizleme senaryosunu simüle etmek amacıyla ham veri havuzuna hatalı formatta (isim anomalileri, boş telefon verisi, mantıksız doğum tarihi) kayıtların eklenmesi.

```
DirtyData.sql  
1 INSERT INTO raw_patients (full_name, phone_number, birth_date, blood_type) VALUES  
2 ('eMiNe bülBül', NULL, '2002-07-27', 'A+'), -- İsim formatı bozuk, Telefon eksik  
3 ('ahmet YILMAZ', '0532 123 4567', '2030-01-01', 'B-'), -- Doğum tarihi mantıksız (Gelecekte)  
4 (' Ayşe Demir ', '05559998877', '1995-05-15', '0+'), -- İsimde gereksiz boşluklar var  
5 ('mehmet kaya', '', '1980-11-20', 'AB+'); -- Telefon numarası boş (Empty string)  
6  
7
```

Şekil 4. Kirli Veri Ekleme

3.2. Veri Dönüştürme ve Temizleme Adımları (Transform)

Teorik Altyapı: Ham verinin, iş kurallarına ve veritabanı kısıtlamalarına (Constraints) uygun hale getirilmesi sürecidir.

Teknik Uygulama: PostgreSQL'in yerleşik string ve mantıksal fonksiyonları kullanılarak kapsamlı bir temizlik katmanı (Transform Layer) tasarlanmıştır:

- **İsim Standartlaştırma:** INITCAP(TRIM(full_name)) kullanılarak gereksiz boşluklar silinmiş ve isimlerin sadece baş harfleri büyük olacak şekilde standart bir formata sokulmuştur.

- **Eksik Veri (NULL) Yönetimi:** COALESCE(NULLIF(phone_number, ''), 'Bilinmiyor') mantığı ile, telefon numarası boş bırakılmış veya NULL gelmiş kayıtlar tespit edilip sistemde hata yaratması engellenmiş, yerine varsayılan bir değer atanmıştır.
- **Mantıksal Doğrulama:** CASE WHEN birth_date > CURRENT_DATE THEN NULL kurgusu ile, doğum tarihi bugünden büyük (gelecekte) olan hatalı kayıtlar karantinaya alınarak izole edilmiştir.

SQL yerleşik fonksiyonları (INITCAP, COALESCE, CASE) kullanılarak kirli veriyi standartlaştıran, eksiklikleri tolere eden ve mantıksız verileri izole eden Transform (Dönüşüm) katmanı kodları.

```
INSERT INTO clean_patients (formatted_name, contact_number, valid_birth_date, blood_type)
SELECT
  -- TRANSFORM 1 (İsim Standartlaştırma): Boşlukları sil (TRIM) ve Baş harfleri büyük yap (INITCAP)
  INITCAP(TRIM(full_name)),

  -- TRANSFORM 2 (Eksik Veri Yönetimi): Telefon numarası NULL veya boş (') ise 'Bilinmiyor' yazdır
  COALESCE(NULLIF(phone_number, ''), 'Bilinmiyor'),

  -- TRANSFORM 3 (Mantıksal Doğrulama): Doğum tarihi gelecekte ise veriyi NULL yaparak karantinaya al
  CASE
    WHEN birth_date > CURRENT_DATE THEN NULL
    ELSE birth_date
  END,

  blood_type
FROM raw_patients;
```

Şekil 5. Veri Transform İşlemleri

3.3. Temizlenmiş Verinin Operasyonel Tabloya Aktarımı (Load)

Teorik Altyapı: Dönüşüm işlemlerinden başarıyla geçen temiz verilerin, hastane bilgi sisteminin asıl tablolarına kalıcı olarak yazılmasıdır .

Teknik Uygulama: Hazırlanan clean_patients hedef tablosuna, yukarıda açıklanan tüm temizleme fonksiyonlarını barındıran INSERT INTO ... SELECT yapısı ile tek bir "Transaction" (işlem) bloğunda veri aktarımı sağlanmıştır.

Transform katmanında temizlenen tüm verilerin tek bir işlem (Transaction) bloğu içerisinde operasyonel hedef tabloya aktarılmasını sağlayan entegre SQL komutu.

```
ExtractTransfromLoad.sql
1 INSERT INTO clean_patients (formatted_name, contact_number, valid_birth_date, blood_type)
2 SELECT
3   -- TRANSFORM 1: İsimlerdeki başındaki/sonundaki boşlukları sil (TRIM) ve Baş harfleri
```

Şekil 6. Veri Yükleme (Load) İşlemi

4. Veri Kalitesi ve Süreç Raporlaması

4.1. Analitik Veri Kalitesi Raporunun (View) Tasarlanması

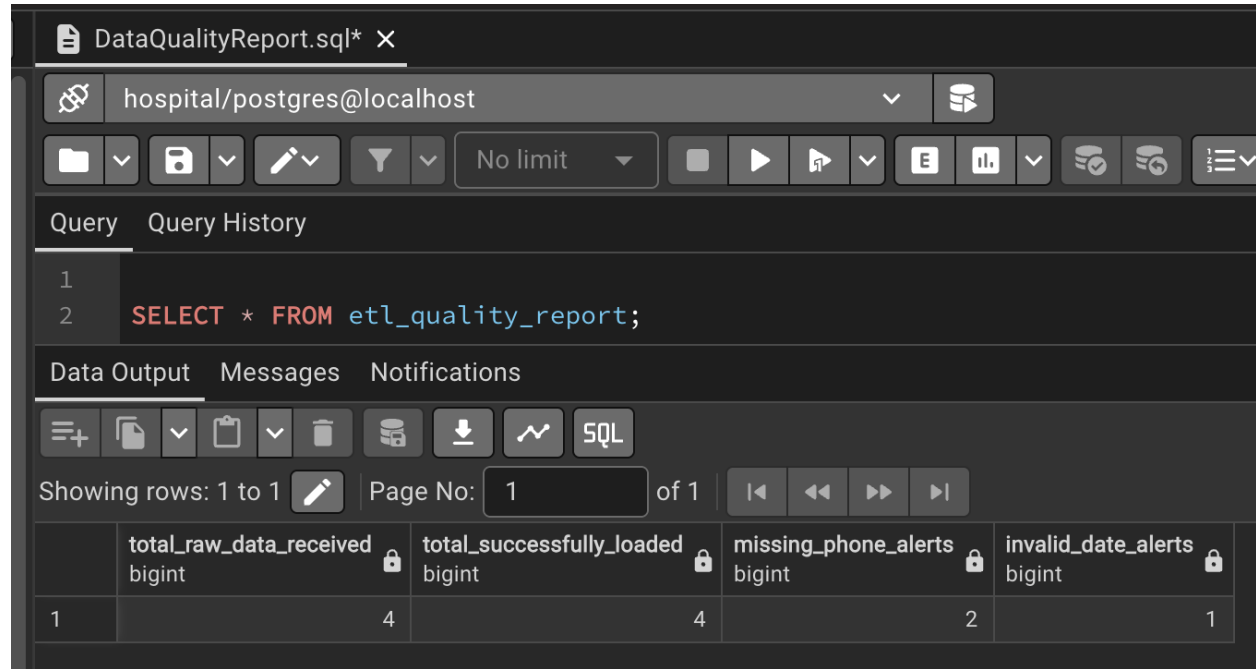
Teorik Altyapı: ETL süreçleri "kara kutu" (black box) olarak çalışmamalı, sistem yöneticilerine hangi verilerin temizlendiği ve hangi verilerde eksiklik saptandığına dair anlık metrikler sunmalıdır.

Teknik Uygulama: Veritabanında etl_quality_report adında bir View (Sanal Tablo) tasarlanmıştır. Bu rapor; dış sistemden toplam kaç ham veri geldiğini, kaçının başarıyla temizlendiğini, kaç hastada iletişim bilgisinin eksik olduğunu ve kaç kayıta mantıksız tarih uyarısı (alert) üretildiğini dinamik olarak hesaplayarak sunmaktadır.

ETL sürecinin şeffaflığını sağlamak amacıyla; veritabanındaki başarılı aktarımları ve saptanan anormallikleri dinamik olarak sayan analitik kalite raporu (View) mimarisi.

```
2  -- Veri temizleme sürecinin başarısını ölçen Analitik Rapor (View)
3
4  CREATE OR REPLACE VIEW etl_quality_report AS
5  SELECT
6      (SELECT COUNT(*) FROM raw_patients) AS total_raw_data_received,
7      (SELECT COUNT(*) FROM clean_patients) AS total_successfully_loaded,
8      (SELECT COUNT(*) FROM clean_patients WHERE contact_number = 'Bilinmiyor') AS missing_phone_alerts,
9      (SELECT COUNT(*) FROM clean_patients WHERE valid_birth_date IS NULL) AS invalid_date_alerts;
10
```

Şekil 6. Rapor Sorgusu



The screenshot shows a database client interface with the following components:

- Query Editor:** Contains the SQL query: `SELECT * FROM etl_quality_report;`
- Results Pane:** Displays the output of the query as a table with 4 columns and 1 row.
- Table Structure:**

	total_raw_data_received bigint	total_successfully_loaded bigint	missing_phone_alerts bigint	invalid_date_alerts bigint
1	4	4	2	1

Şekil 7. ETL Veri Kalitesi (Quality Report) Analiz Ekranı

5. Sonuç ve Kazanımlar

Gerçekleştirilen ETL mimarisi projesiyle, hastane sistemine girebilecek kirli ve yapılandırılmamış veriler uygulama (Application) katmanına gelmeden veritabanı (Database) seviyesinde durdurulmuş ve standartlaştırılmıştır. Geliştirilen SQL tabanlı veri temizleme scriptleri sayesinde manuel düzeltme ihtiyacı ortadan kaldırılmış, veri güvenilirliği maksimize edilmiş ve Veri Kalitesi Raporları (Quality Views) ile sürecin tamamen denetlenebilir olması sağlanmıştır.